

Q-Shield

Lapisan verifikasi yang mengikat rekening merchant di dalam kode QRIS dengan lokasi fisik tempat kode itu ditempel — dijalankan sebelum PIN dimasukkan.

01 Masalah

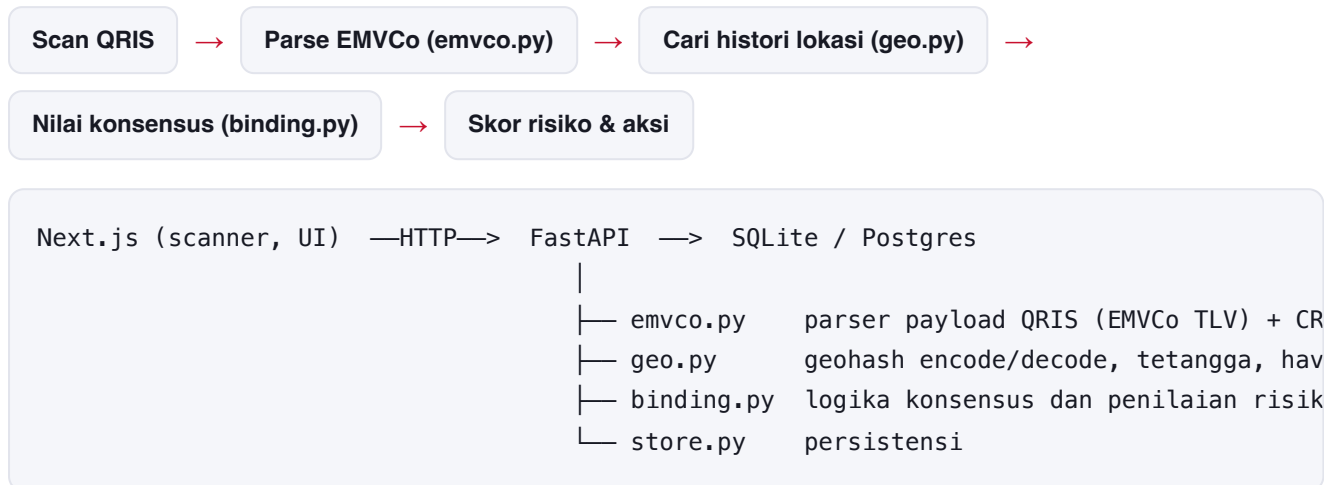
QRIS memverifikasi rekening merchant yang tertanam dalam sebuah kode — tapi tidak satu pun lapisan dalam stack yang memverifikasi ikatan antara rekening itu, artefak fisik yang menampilkannya, dan lokasi tempat artefak itu dipasang.

Konsekuensinya: stiker QRIS palsu adalah payload yang sah secara sintaksis. CRC-nya valid, formatnya benar, nama merchant bisa ditiru persis. Tidak ada yang bisa membedakannya dari yang asli tanpa mengetahui merchant mana yang seharusnya berada di lokasi itu — dan tidak ada satu pihak pun yang saat ini memegang informasi tersebut.

Modus yang dicegah: pelaku menempelkan stiker QRIS palsu di atas stiker asli sebuah warung. Nama merchant di layar pembayaran tetap terlihat benar — hanya nomor rekening tujuan di baliknya yang berbeda. Korban tidak punya cara memeriksa ini secara manual.

02 Cara Kerja

Q-Shield diposisikan sebagai SDK di dalam aplikasi pembayaran (bukan aplikasi tersendiri) yang memeriksa QRIS **sebelum** PIN dimasukkan.



Setiap lokasi yang pernah dilihat sistem menjadi **binding**: pasangan NMID (ID merchant) dan titik koordinat, dengan riwayat jumlah pengamat independen. Saat QR baru dipindai, sistem mencari binding terdekat lewat indeks geohash presisi 7 ($\pm 152 \times 153$ m) ditambah 8 sel tetangganya, lalu memvalidasi kecocokan sebenarnya dengan jarak haversine radius 50 m — bukan sekadar kesamaan geohash, karena pergeseran GPS 20 m saja mengubah geohash presisi 8 pada 81,9% kasus (diukur lewat `calibrate_geo.py`).

Empat tingkat respons — bukan blokir biner

SKOR RISIKO	AKSI	ARTI
0–25	PROCEED	Aman dilanjutkan, konsisten dengan histori
26–50	WARN	Lokasi belum/tidak sepenuhnya dikenal
51–75	STEP_UP	Perlu verifikasi tambahan (biometrik)
76–100	COOLING_OFF	Pembayaran ditahan 30 menit

Alasannya ekonomis, bukan cuma teknis: false positive pada sistem pembayaran itu mahal. Cooling-off dipakai sebagai respons risiko tertinggi karena rekayasa sosial selalu bergantung pada tekanan waktu — menunda lebih efektif daripada menolak langsung.

03 Keputusan Desain Kunci

1 "Belum dikenal" bukan "aman"

Sistem tidak pernah menyatakan binding aman hanya karena belum ada konflik. Statusnya `verified` / `unknown` / `anomaly` — analoginya, antivirus tidak mengklaim berkas tak dikenal itu bersih. Ini menjawab masalah *cold start*: hari pertama sistem berjalan, database kosong, dan menyatakan "aman" akan berbahaya.

2 Bobot sinyal berskala dengan kekuatan bukti

Konflik NMID pada binding dengan 47 pengamat adalah bukti jauh lebih kuat daripada konflik pada binding yang baru mencapai ambang minimum. Bobotnya dihitung $60 + \min(25, \text{observer_count} // 2)$ — bukan nilai datar.

3 Merchant bersebelahan ≠ pertukaran stiker

Dua merchant sah berjarak 8 m (ruko, food court) tidak boleh saling memicu alarm. Kuncinya: pertukaran stiker membuat binding lama *berhenti terlihat*; kalau dua NMID sama-sama mapan dan masih aktif di jangkar yang sama, itu koeksistensi (sinyal `adjacent_merchant`, bobot ringan) — bukan pemalsuan.

4 Privasi tertanam di skema, bukan kebijakan

Tidak ada kolom `user_id` di skema mana pun. Tabel `observations` tidak menyimpan koordinat, sehingga tidak bisa dipakai merekonstruksi pergerakan seseorang — yang disimpan adalah **ikatan** (properti lokasi merchant), bukan **kunjungan** pengguna.

5 Anomali tidak membangun reputasi

Pemindaian yang menghasilkan `anomaly` tidak dicatat sebagai pengamatan — supaya penyerang tidak bisa mencemari basis data dengan memindai stikernya sendiri berulang kali sampai binding palsu terlihat mapan. Diuji: 5 pemindaian anomali berturut-turut tidak menambah satu pun binding.

04 Hasil Pengujian

13 SKENARIO BINDING DIUJI	2,8 ms LATENSI P50	3,5 ms LATENSI P95
-------------------------------------	------------------------------	------------------------------

200 permintaan berturut-turut lewat SQLite lokal: p50 2,8 ms, p95 3,5 ms, maks 22,4 ms — jauh di bawah anggaran 200 ms. Empat berkas pengujian (`test_emvco.py`, `test_geo.py`, `test_binding.py`, `test_api.py`) mencakup parser payload, geohash & jarak, 13 skenario konsensus (termasuk kasus ruko dan relokasi merchant), serta alur end-to-end lewat API sungguhan. Angka latensi ini belum termasuk jaringan dan belum diuji pada volume produksi.

05 Arah Pengembangan

Ini PoC Layer 1 (verifikasi lokasi). Batasan yang sudah diketahui sekaligus menjadi peta jalan berikutnya:

Presisi lokasi

jangka pendek

Ambient WiFi fingerprinting

Merchant berjarak <15 m sulit dibedakan lewat GPS saja. Tidak tersedia lewat browser — karena itu PoC ini berbasis web, dan MVP-nya memerlukan SDK native.

Integritas perangkat

jangka pendek

Deteksi mock location

GPS palsu dapat mencemari basis data binding. Mitigasinya adalah deteksi integritas perangkat — alasan teknis mengapa Layer 2 diperlukan.

Layer 2

Behavioral scoring untuk transfer manual

jangka menengah

Q-Shield saat ini menutup jalur QRIS. Modus rekayasa sosial lewat transfer bank manual masih berupa rancangan, belum diimplementasikan.

Onboarding merchant

jangka menengah

Pendaftaran mandiri & jalur konfirmasi

Mengatasi cold start secara penuh, serta menangani relokasi merchant yang sah (saat ini memicu satu kali peringatan) dan merchant keliling (belum ditangani sama sekali, perlu penandaan khusus saat pendaftaran).

Kalibrasi lapangan

berkelanjutan

Parameter dari data nyata, bukan asumsi demo

`ANCHOR_RADIUS_M` , `MIN_OBSERVERS` , `MIN_AGE_HOURS` , `SCATTER_MIN_KM` , dan `STALE_DAYS` semuanya sudah diekspos terpisah di `binding.py` sebagai titik awal demo — perlu dikalibrasi ulang dari data lapangan sebelum produksi.

Berbagi data

jangka panjang

Kolaborasi antar-PJP tanpa melanggar kerahasiaan

Skema privasi (tanpa `user_id` , tanpa koordinat kunjungan) dirancang supaya ikatan merchant-lokasi bisa dipertukarkan antar penyelenggara jasa pembayaran tanpa melanggar kerahasiaan bank maupun UU PDP.